Lab 7 : Text Similarity Measures — Cosine, Jaccard, and WordNet-based Similarity

```
#Text similarty methods jaccard,cosine,wordnet
```

```
documents = [
    "Machine learning improves data analysis",
    "Machine learning enhances data analytics",
    "Artificial intelligence helps machines learn",
    "AI and machine learning are related fields",
    "Football is a popular sport worldwide",
    "Cricket is played by many countries",
    "Healthy food improves human health",
    "Eating nutritious food keeps the body fit",
    "Programming requires logic and practice",
    "Coding needs logical thinking and patience",
    "The sun rises in the east",
    "Dogs are loyal animals",
    "Cats are independent pets",
    "Technology is evolving rapidly",
    "Modern technology changes the world"
]
```

STEP 1: TEXT PREPROCESSING

```
import nltk
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Download required NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation marks
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Tokenize text into words
    tokens = word_tokenize(text)

    # Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]

    # Lemmatize words (convert to base form)
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    return tokens
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

STEP 2: FEATURE REPRESENTATION

```
import nltk
nltk.download('punkt_tab', quiet=True)

from sklearn.feature_extraction.text import TfidfVectorizer

# Convert documents into TF-IDF vectors for cosine similarity
tfidf_vectorizer = TfidfVectorizer(
```

```
    preprocessor=lambda x: ' '.join(preprocess(x))
)

tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
```

STEP 3: COSINE SIMILARITY

```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

cos_sim = cosine_similarity(tfidf_matrix)

cosine_df = pd.DataFrame(
    cos_sim,
    index=[f"Doc {i+1}" for i in range(len(documents))],
    columns=[f"Doc {i+1}" for i in range(len(documents))]
)

print(cosine_df.round(2))

pairs = []

for i in range(len(documents)):
    for j in range(i + 1, len(documents)):
        pairs.append((f"Doc {i+1}", f"Doc {j+1}", cos_sim[i][j]))

pairs = sorted(pairs, key=lambda x: x[2], reverse=True)[:15]

print("Top 15 Most Similar Document Pairs:\n")
for d1, d2, score in pairs:
    print(f"{d1} - {d2} : {score:.3f}")
```

```
        Doc 1  Doc 2  Doc 3  Doc 4  Doc 5  Doc 6  Doc 7  Doc 8  Doc 9  Doc 10  \
Doc 1    1.00   0.50   0.12   0.28    0.0    0.0   0.19   0.00    0.0     0.0
Doc 2    0.50   1.00   0.12   0.28    0.0    0.0   0.00   0.00    0.0     0.0
Doc 3    0.12   0.12   1.00   0.12    0.0    0.0   0.00   0.00    0.0     0.0
Doc 4    0.28   0.28   0.12   1.00    0.0    0.0   0.00   0.00    0.0     0.0
Doc 5    0.00   0.00   0.00   0.00    1.0    0.0   0.00   0.00    0.0     0.0
Doc 6    0.00   0.00   0.00   0.00    0.0    1.0   0.00   0.00    0.0     0.0
Doc 7    0.19   0.00   0.00   0.00    0.0    0.0   1.00   0.15    0.0     0.0
Doc 8    0.00   0.00   0.00   0.00    0.0    0.0   0.15   1.00    0.0     0.0
Doc 9    0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    1.0     0.0
Doc 10   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     1.0
Doc 11   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     0.0
Doc 12   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     0.0
Doc 13   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     0.0
Doc 14   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     0.0
Doc 15   0.00   0.00   0.00   0.00    0.0    0.0   0.00   0.00    0.0     0.0

        Doc 11  Doc 12  Doc 13  Doc 14  Doc 15
Doc 1      0.0     0.0     0.0    0.00    0.00
Doc 2      0.0     0.0     0.0    0.00    0.00
Doc 3      0.0     0.0     0.0    0.00    0.00
Doc 4      0.0     0.0     0.0    0.00    0.00
Doc 5      0.0     0.0     0.0    0.00    0.00
Doc 6      0.0     0.0     0.0    0.00    0.00
Doc 7      0.0     0.0     0.0    0.00    0.00
Doc 8      0.0     0.0     0.0    0.00    0.00
Doc 9      0.0     0.0     0.0    0.00    0.00
Doc 10     0.0     0.0     0.0    0.00    0.00
Doc 11     1.0     0.0     0.0    0.00    0.00
Doc 12     0.0     1.0     0.0    0.00    0.00
Doc 13     0.0     0.0     1.0    0.00    0.00
Doc 14     0.0     0.0     0.0    1.00    0.23
Doc 15     0.0     0.0     0.0    0.23    1.00
Top 15 Most Similar Document Pairs:

Doc 1 - Doc 2 : 0.496
Doc 1 - Doc 4 : 0.285
Doc 2 - Doc 4 : 0.276
Doc 14 - Doc 15 : 0.234
Doc 1 - Doc 7 : 0.187
Doc 7 - Doc 8 : 0.148
Doc 1 - Doc 3 : 0.123
Doc 2 - Doc 3 : 0.119
Doc 3 - Doc 4 : 0.115
Doc 1 - Doc 5 : 0.000
Doc 1 - Doc 6 : 0.000
Doc 1 - Doc 8 : 0.000
```

```
Doc 1 - Doc 9 : 0.000
Doc 1 - Doc 10 : 0.000
Doc 1 - Doc 11 : 0.000
```

STEP 4: JACCARD SIMILARITY

```
def jaccard_similarity(doc1, doc2):
    # Convert documents to sets of words
    set1 = set(preprocess(doc1))
    set2 = set(preprocess(doc2))

    # Calculate Jaccard similarity
    intersection = set1.intersection(set2)
    union = set1.union(set2)

    return len(intersection) / len(union)

print("\nJaccard Similarity Scores:")
for i in range(n):
    for j in range(i + 1, n):
        score = jaccard_similarity(documents[i], documents[j])
        print(f"Doc {i} & Doc {j} -> Score: {score:.3f}")
```

```
Jaccard Similarity Scores:
Doc 0 & Doc 1 -> Score: 0.429
Doc 0 & Doc 2 -> Score: 0.111
Doc 0 & Doc 3 -> Score: 0.250
Doc 0 & Doc 4 -> Score: 0.000
Doc 0 & Doc 5 -> Score: 0.000
Doc 0 & Doc 6 -> Score: 0.111
Doc 0 & Doc 7 -> Score: 0.000
Doc 0 & Doc 8 -> Score: 0.000
Doc 0 & Doc 9 -> Score: 0.000
Doc 0 & Doc 10 -> Score: 0.000
Doc 0 & Doc 11 -> Score: 0.000
Doc 0 & Doc 12 -> Score: 0.000
Doc 0 & Doc 13 -> Score: 0.000
Doc 0 & Doc 14 -> Score: 0.000
Doc 1 & Doc 2 -> Score: 0.111
Doc 1 & Doc 3 -> Score: 0.250
Doc 1 & Doc 4 -> Score: 0.000
Doc 1 & Doc 5 -> Score: 0.000
Doc 1 & Doc 6 -> Score: 0.000
Doc 1 & Doc 7 -> Score: 0.000
Doc 1 & Doc 8 -> Score: 0.000
Doc 1 & Doc 9 -> Score: 0.000
Doc 1 & Doc 10 -> Score: 0.000
Doc 1 & Doc 11 -> Score: 0.000
Doc 1 & Doc 12 -> Score: 0.000
Doc 1 & Doc 13 -> Score: 0.000
Doc 1 & Doc 14 -> Score: 0.000
Doc 2 & Doc 3 -> Score: 0.111
Doc 2 & Doc 4 -> Score: 0.000
Doc 2 & Doc 5 -> Score: 0.000
Doc 2 & Doc 6 -> Score: 0.000
Doc 2 & Doc 7 -> Score: 0.000
Doc 2 & Doc 8 -> Score: 0.000
Doc 2 & Doc 9 -> Score: 0.000
Doc 2 & Doc 10 -> Score: 0.000
Doc 2 & Doc 11 -> Score: 0.000
Doc 2 & Doc 12 -> Score: 0.000
Doc 2 & Doc 13 -> Score: 0.000
Doc 2 & Doc 14 -> Score: 0.000
Doc 3 & Doc 4 -> Score: 0.000
Doc 3 & Doc 5 -> Score: 0.000
Doc 3 & Doc 6 -> Score: 0.000
Doc 3 & Doc 7 -> Score: 0.000
Doc 3 & Doc 8 -> Score: 0.000
Doc 3 & Doc 9 -> Score: 0.000
Doc 3 & Doc 10 -> Score: 0.000
Doc 3 & Doc 11 -> Score: 0.000
Doc 3 & Doc 12 -> Score: 0.000
Doc 3 & Doc 13 -> Score: 0.000
Doc 3 & Doc 14 -> Score: 0.000
Doc 4 & Doc 5 -> Score: 0.000
Doc 4 & Doc 6 -> Score: 0.000
Doc 4 & Doc 7 -> Score: 0.000
Doc 4 & Doc 8 -> Score: 0.000
Doc 4 & Doc 9 -> Score: 0.000
Doc 4 & Doc 10 -> Score: 0.000
```

STEP 5: WORDNET SEMANTIC SIMILARITY

```
from nltk.corpus import wordnet as wn

def wordnet_sentence_similarity(sent1, sent2):
    score = 0
    count = 0

    # Compare each word in sentence 1 with sentence 2
    for word1 in preprocess(sent1):
        synsets1 = wn.synsets(word1)
        if not synsets1:
            continue

        for word2 in preprocess(sent2):
            synsets2 = wn.synsets(word2)
            if not synsets2:
                continue

            # Use path similarity between synsets
            similarity = synsets1[0].path_similarity(synsets2[0])
            if similarity is not None:
                score += similarity
                count += 1

    # Return average similarity score
    return score / count if count != 0 else 0

print("\nWordNet Semantic Similarity (Sample Pairs):")
for i in range(10):  # Apply to at least 10 sentence pairs
    score = wordnet_sentence_similarity(documents[i], documents[i + 1])
    print(f"Doc {i} & Doc {i+1} -> Score: {score:.3f}")
```

```
WordNet Semantic Similarity (Sample Pairs):
Doc 0 & Doc 1 -> Score: 0.226
Doc 1 & Doc 2 -> Score: 0.152
Doc 2 & Doc 3 -> Score: 0.143
Doc 3 & Doc 4 -> Score: 0.102
Doc 4 & Doc 5 -> Score: 0.132
Doc 5 & Doc 6 -> Score: 0.115
Doc 6 & Doc 7 -> Score: 0.134
Doc 7 & Doc 8 -> Score: 0.096
Doc 8 & Doc 9 -> Score: 0.117
Doc 9 & Doc 10 -> Score: 0.086
```

STEP 6: COMPARISON & ANALYSIS

Comments for report:

- Cosine similarity performs best for copied and slightly modified documents
- Jaccard similarity depends heavily on word overlap and fails for paraphrasing
- WordNet captures semantic similarity but is slower and less effective for long texts
- Lexical methods are faster; semantic methods are more meaningful

STEP 7 : Lab Report Section

This experiment demonstrated that no single similarity method is ideal for all scenarios. Cosine similarity is robust and widely used for document comparison, Jaccard similarity is simple but limited to exact word overlap, and WordNet-based similarity is useful for detecting semantic relationships. The choice of similarity metric depends on the nature of the text and the application. Combining lexical and semantic approaches can provide more accurate and reliable text similarity analysis in real-world NLP applications.